

AI AGENT 项目实战营 · 第 06 节

多工具协同与错误恢复

单把工具是玩具，多把协同才是产品

本节是项目课（60 分钟）。下课时你交出 P2 —— 给 Agent 接上至少 1 个本地工具 + 1 个 MCP 工具，并让它们协同、会兜错。

PROJECT · ResuMatch · PROJECT CLASS · 60 MIN



一句话，全自动跑完

你只说 『帮我看看这个 JD，我合不合适？』

① 读 JD

抽出硬性要求



② 搜真实岗位

打公开接口



③ 算匹配分

代码算，不猜



④ 给改写建议

收口成结果

OUTPUT 一份完整匹配报告（含分数、缺口、改写句）。你只说了一句 —— 拆步、连调、收尾全是它自己做的，这就是 P2 的样子。

从一把到一条链

单把工具 · 玩具

- 只会查一件事
- 一问一答
- 缺一步就断
- 像个查询框

多把协同 · 真产品

- 一句话触发多步
- 上一步喂下一步
- 中途失败还能走完
- 交出完整结果

前两节你学会造一把、接一个 MCP；今天学的是把它们串起来干活。

今天七步交 P2

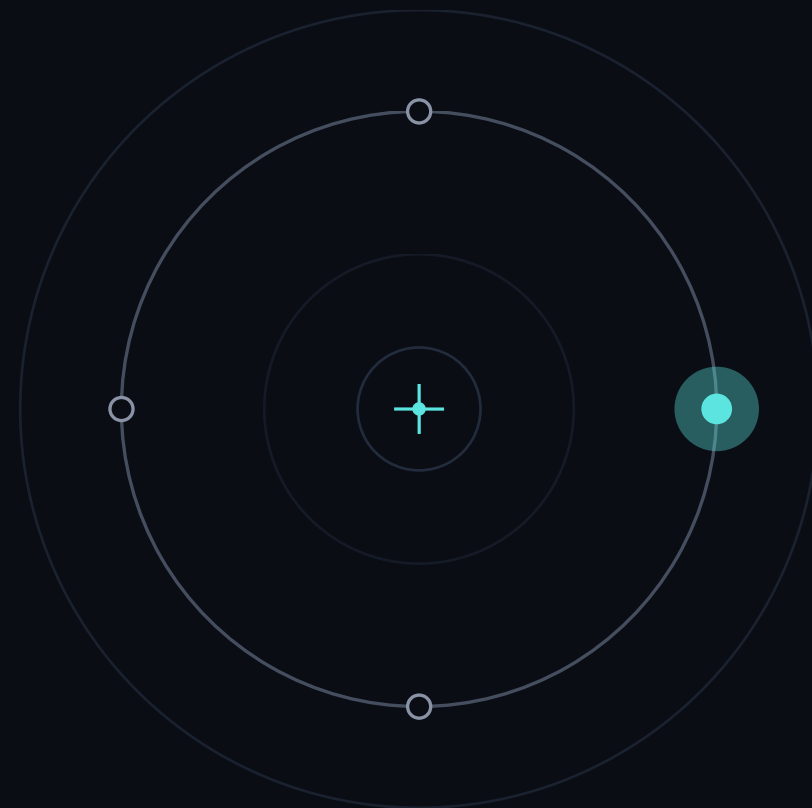


每一步只讲一个新东西，且都跑给你看。第 ⑥ 站是今天的落点。

01

PART 1 · WHAT GOOD LOOKS LIKE

上限长什么样



四条达标线



多步端到端跑通

一句话触发、连调几把、给出完整结果



用真实数据

至少一把工具接真数据源，不是全写死



不编造

答案有据，数字用代码算，不让模型猜



中途某步失败不崩

一步挂了，链条能降级走完或如实交代

这四条是每个项目的底线，P2 只是它在多工具场景的第一次落地。

三种项目都成立



求职助手 ResuMatch

读 JD · 搜岗位 · 算分 · 改写



课程 RAG 助手

检索 · 总结 · 出题



个人日程助手

查日历 · 建事件 · 发提醒

形态不同，套路一样 —— 几把工具串起来，端到端跑通。今天 demo 走 ResuMatch，另两条留给选它的同学照搬。

P2 达标四条



至少 1 个本地工具

你写的 `@function_tool`（读作『函数工具』）



至少 1 个 MCP 工具

接一个 MCP Server（模型接外部服务的标准接口）



能协同

一段对话里两类工具都被用到、能串起来



会兜错

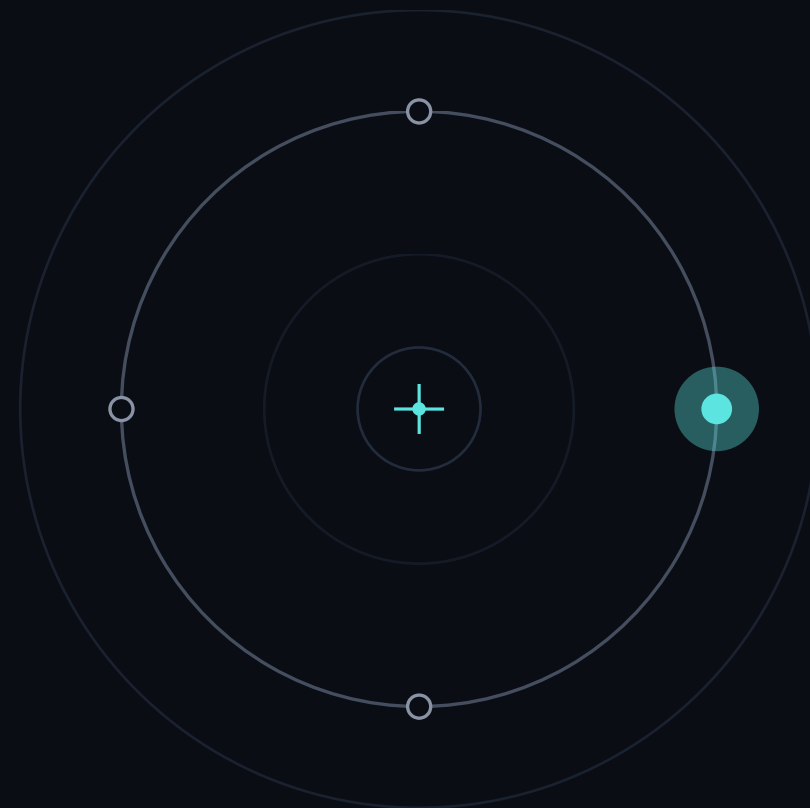
工具报错时不让整轮崩

交付物很简单 —— 一段能跑的对话 + 代码。评分看：工具设计 / 错误处理 / 协同质量。

02

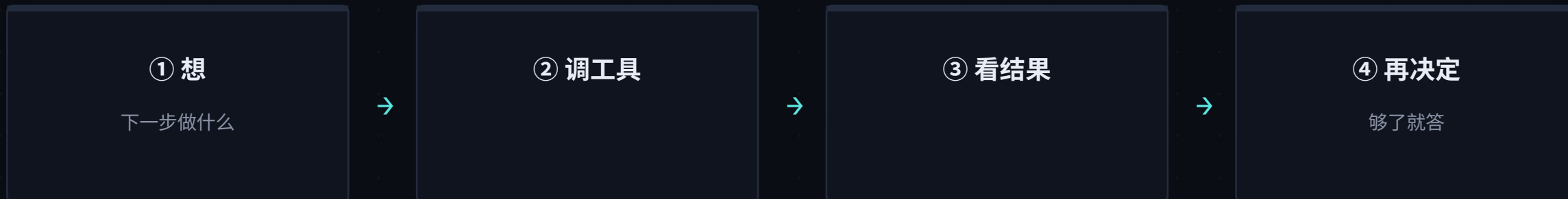
PART 2 · REACT LOOP

ReAct 多步循环



ReAct 转的圈

ReAct = 边想边做。术语先落地，下面就是它转的圈。



关键 每转一圈 = 想一次 + 调一把工具；任务复杂就多转几圈。Runner 替你转这个圈，你不用自己写循环。

数它走了几步

demo 02 · ResuMatch 一句话连调三把，打印它走过的每一步。

```
result = Runner.run_sync(agent, ask, max_turns=10)
for item in result.new_items: # 走过的每一步
    kind = type(item).__name__
    if kind == "ToolCallItem": # 请求调工具
        print("调用", item.raw_item.name)
    elif kind == "ToolCallOutputItem": # 结果回灌
        print("返回", item.output)
```

你会看到

调用 parse_jd

返回 {...}

调用 score_resume

返回 {...}

调用 suggest_rewrite

返回 {...}

作答 (最后一句)

上一把喂下一把

结构化输出 = 返回字典（dict），而不是一句话。字段稳定，下一把就能精准取用。



关键 让工具返回带稳定字段的 dict，下一把就能精准取用，不靠模型猜。（返回值怎么设计 L4 讲过，这里只讲字段在链条里怎么流。）

一个字段接起来

demo 03 · 关键就一处 —— 上一把的 `must_have` 成了下一把的入参。

```
@function_tool
def parse_jd(jd_text: str) → dict:
    """ 读 JD，抽出硬性要求（供打分用）。 """
    return {"role": role, "must_have": must_have}    # 稳定字段

@function_tool
def score_resume(resume_text: str, must_have: list[str]) → dict:
    """ 按 must_have 给简历算分、列缺口。 """
    return {"score": score, "missing": missing}
```

字段名对齐（都叫 `must_have`），模型就知道该把谁传给谁。

排队还是并行

排队 • 后一步要前一步的结果

— parse_jd → score_resume → suggest_rewrite

— score 要先拿到 must_have，只能等

并行 • 互不依赖可同时做

— 读 JD 和 搜岗位 谁先都行

— 不依赖彼此，可同时发

判断依据只有一句 后一步是否需要前一步的输出。需要就排队，不需要就能并行。

每转一圈一次调用



每转一圈 = 一次模型调用

更慢、更贵



三把工具的任务

通常转 3-4 圈，一次问答就是好几次调用



所以

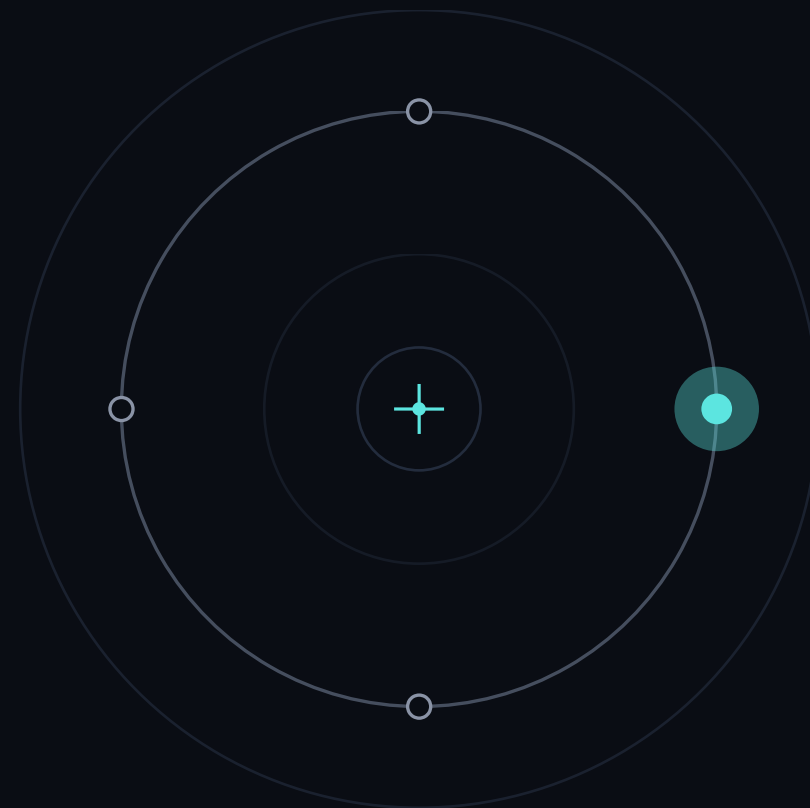
能合并的步骤合并，不必要的工具别挂

多步很强，但不是免费的 —— 链条越长，账单和延迟越长，够用就好。

03

PART 3 · RECOVERY IN A CHAIN

链条里的错误恢复



回顾：三层防线

第 4 节你已经会了单把工具的三层防线：

- ① 工具内 try/except —— 把异常变成友好错误返回
- ② 瞬时错误退避重试（等一会儿再试）
- ③ Runner 设 max_turns 兜底，防死循环

今天要补的 这些直接用，不再讲原理。

新问题：它们在『一条链』里怎么配合 —— 链条中间某步挂了，怎么让它走完（见下一页）。

一步挂，别全崩

没兜错 · 一步挂全崩

- search_jobs 被限流 429（请求太频被拒）
- 裸异常抛进循环
- score / suggest 全没跑
- 用户拿到半截报错

兜住了 · 降级走完

- 429 被工具接住
- 降级用离线样例、标注来源
- 链条继续 打分 / 改写
- 交出完整报告（诚实说明是样例）

核心 让失败的那一步『降级返回』，而不是『抛异常』，后面几步才能接着跑。

限流就降级继续

demo 04 · search_jobs 打真实 HN 接口；被 429 或超时就降级，链条不断。

```
@function_tool
def search_jobs(keyword: str) → dict:
    try:
        r = httpx.get(HN_API, params=..., timeout=6)
        r.raise_for_status()                # 429/5xx 抛出
        return {"source": "hn", "jobs": _clean(r.json())}
    except httpx.HTTPError as e:           # 不 raise，降级返回
        return {"source": "offline-sample", "jobs": OFFLINE}
```

它不 raise，而是 return 一个带 source 的 dict；模型据此照常往下打分、改写，并在报告里说明这是样例。

降级可以，编造不行

1

降级 $\neq$ 编造

换用样例数据可以，但必须如实告诉用户『这是样例，不是实时』

2

数字只能来自工具返回值

模型不许自己填（score 用代码算，L3/L4 讲过的红线）

3

一步失败就说清

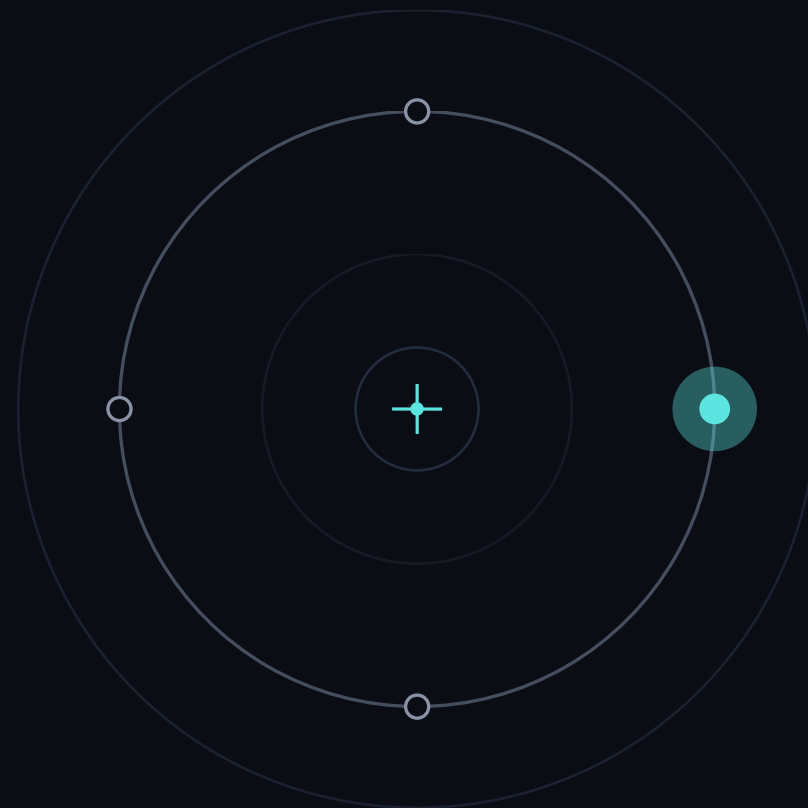
哪步失败、用了什么兜底

会失败还诚实，比假装成功更值钱 —— 这正是达标线里的『不编造』落到链条上。

04

PART 4 · DELIVER P2

交付 P2



三块拼一个 Agent

模板路径 demo_code/05_p2_assistant.py — 你 P2 的起点。

① 本地工具
@function_tool 你自己写

+

② MCP 工具
接 filesystem Server

+

③ 记忆
SQLiteSession 多轮

```
agent = Agent(  
    name="ResuMatch", instructions=INSTRUCTIONS, # 换成你的角色  
    tools=[parse_jd, score_resume, suggest_rewrite], # 本地  
    mcp_servers=[fs_server], # MCP：报告落盘  
)
```

离线友好 缺 Node / 缺 mcp 包 / 缺 Key 会自动降级为『仅本地工具』，照样跑不崩。

五个 demo 走查

01

两把本地工具

按需选对

02

一句话多步

看 ReAct 转圈

03

工具间传值

字段对齐

04

链中失败

降级走完

05

本地 + MCP

你的 P2 起点

按 01→05 顺序跑，每个只加一个新概念；课上精简走查，细节课后自己读。不逐行念代码。

四把工具自己接力

demo 05 · 一句话触发整条 ResuMatch 链，最后把报告交给 MCP 落盘。

```
ask = "这是我的简历和这段 JD，  
我合适吗？帮我出份报告。"  
result = await Runner.run(  
    agent, ask, session=session,  
    max_turns=12)  
  
print(result.final_output)
```

它自己走的步

parse_jd

search_jobs

score_resume

suggest_rewrite

[MCP] write_file(report.md)

作答

前四把本地工具算出报告，最后一步交 MCP 写文件 —— 这就是 P2 要的『本地 + MCP 协同』。

动手：接两把工具

T1 起步

跑通 01-04 看懂四件事；把 05 的本地工具换成你选题的一把，能协同一次

T2 进阶

本地一把 + MCP 一把都接上，一段对话里两类都被调用到（达 P2 硬线）

T3 挑战

串更长的链（ ≥ 3 把接力）+ 打印 ReAct 轨迹 + 故意让一步失败看它降级走完

MCP 接不上不要慌 —— 05 会自动降级，先把本地 + 多步 + 恢复跑顺，MCP 课后补。

P2 自检五条

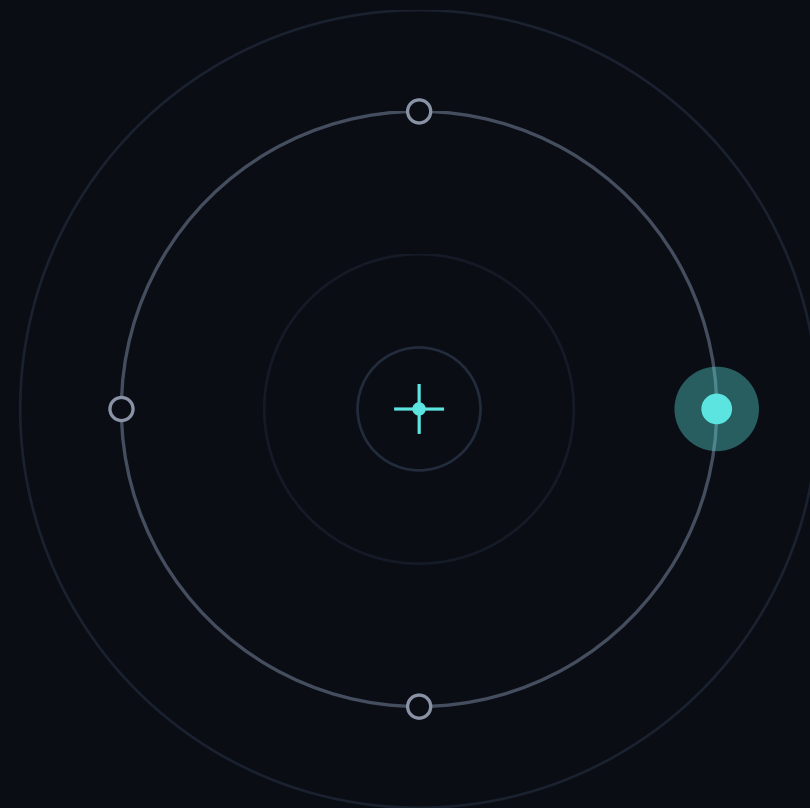
- ✓ 至少 1 个本地工具（你写的）
- ✓ 至少 1 个 MCP 工具（接上一个 Server）
- ✓ 一段对话里两类都被调用到
- ✓ 至少一步失败能兜住不崩
- ✓ 有一段能跑的对话 + 代码可交

五条全勾 = P2 达标。交付这周（W4）：代码 + 一段能跑的对话。

05

PART 5 · FRAMEWORK VIEW

框架视野



三种编排怎么选

术语 编排 = 安排多步怎么走；状态机 = 按固定图走步骤。



ReAct 自由循环

SDK · 你现在用的

模型自己决定下一步，最灵活，适合探索型任务



状态机 LangGraph

步骤画成固定图

可回退可审计，适合流程稳定、要可控的场景



低代码 n8n

拖拽连节点

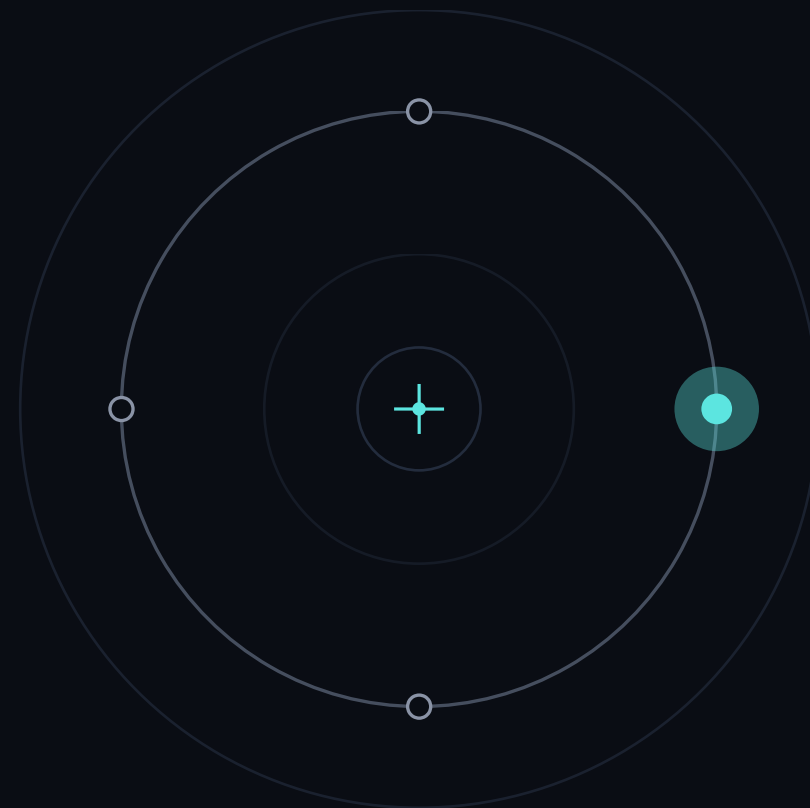
不写代码，适合日程助手这类自动化编排

何时升级 链条变长、要可审计 / 团队维护，从自由循环升到状态机；纯自动化、少写代码就用 n8n。

06

PART 6 · WRAP UP

收尾



选题两轨

轨道 A · 跟旗舰 ResuMatch

- 直接照搬今天的链：读 JD+ 搜岗位 + 算分 + 改写
- 把 05 模板改成自己的数据
- 上手快、有完整参照

轨道 B · 自选题

- 课程 RAG / 日程助手 / 你的点子
- 一套同一路
- 几把工具串起来端到端跑通

不管哪轨，都要过那四条达标线；选题这周定下来，P2 就按选题做。

作业两档

基础 · 必做

- P2 接 2 把工具能协同 (≥ 1 本地 + ≥ 1 MCP)
- 演示 1 次链中某步失败能恢复走完
- 交代码 + 一段能跑的对话

进阶 ★ · 加分

- 更长的工具链 (≥ 3 把接力)
- 写操作做成幂等 (同名文件先查再写)
- 用一句话 eval 证明真跑通

幂等 = 重复做一次结果不变。基础保过线，进阶练的是生产级习惯。

下节：加记忆

① 今天你的 Agent 学会三件事

选对工具、把工具串起来、一步失败也能恢复

② 但它现在关掉就忘

多轮之间记不住你刚说过的话

课前必看 《Demo · 会话状态与窗口裁剪》15 分钟。这周交 P2（本地 + MCP）。下节见。

消息历史、滑动窗口、Summary 压缩，让它记住上下文